

KaMoTech

Project Management Plan

Smart Goat Farm Management System



Plan version	1.0
Baseline date	August 14, 2026
Delivery window	August 17 - October 9, 2026
Plan owner	Project Lead
Status	Execution / integration and validation

A delivery, quality, risk, and release framework for the integrated capstone system.

1. Purpose

This plan guides the completion, integration, validation, and presentation of KaMoTech, a Django-based smart goat farm platform. It coordinates the web application, IoT devices, automated feeding, computer vision, alerts, analytics, SMS, security monitoring, and goat marketplace into one defensible capstone deliverable.

The plan uses an eight-week delivery window because no fixed submission date or named team roster is recorded in the repository. Dates and role assignments should be re-baselined when those details are confirmed.

2. Project Objectives

By October 9, 2026, the project should:

1. Deliver a stable end-to-end farm monitoring workflow from sensor/device input to dashboard, alerts, and reports.
2. Demonstrate reliable automated feeding with schedules, logs, safety controls, and physical-device feedback.
3. Demonstrate goat and person detection using the approved ML models and camera source.
4. Provide accurate goat records, analytics, notifications, and marketplace workflows with appropriate access control.
5. Pass agreed functional, integration, security, recovery, and user-acceptance tests.
6. Produce the documentation, deployment package, evidence, and presentation needed for capstone evaluation.

3. Current Baseline

As of August 14, 2026:

- Django system validation passes with no reported issues.
- No uncommitted model changes are detected by `makemigrations --check --dry-run`.
- All 19 existing automated tests pass (analytics, marketplace, and IoT sensor API).
- Implemented application areas include dashboard, IoT, feeding, ML detection, analytics, security, SMS, and marketplace.
- ESP32 firmware variants exist for sensors, servo feeding, Django integration, and MQTT integration.
- Goat detection is documented and supported by YOLO; individual goat recognition remains an optional or future capability unless explicitly required by the panel.
- The automated-feeding code still contains TODOs for production MQTT publication to the physical feeder.
- Development uses an in-memory WebSocket channel layer; production-grade real-time operation requires Redis or an explicitly accepted alternative.
- Automated test coverage is concentrated in analytics, marketplace, and selected IoT endpoints; feeding, ML, security, SMS, dashboard, WebSockets, and complete hardware workflows need stronger coverage.
- No Git repository metadata was available in this workspace at baseline, so source-control history and release traceability could not be verified.

4. Scope

In scope for the capstone release

- User authentication and role-based access.
- Main farm dashboard and operational status views.
- ESP32 registration, sensor ingestion, and sensor visualization.
- Indoor environmental monitoring, including light and air-quality readings.
- Feeder scheduling, manual actuation, feed-level tracking, and feed logs.
- Camera connectivity and goat/person detection.
- Alerts through the application and configured SMS channel.
- Goat records, detection history, behavior/grass-health information where used by the demonstrated workflow.
- Analytics, reports, and export capability.
- Marketplace listing, inquiry, reservation, pickup, completion, and access-control workflows.
- Deployment configuration, backup/recovery procedure, operator guide, test evidence, and defense/demo package.

Out of scope unless promoted through change control

- Native Android or iOS applications.
- Commercial-scale multi-farm tenancy.
- Online payment processing and shipping logistics.
- Fully autonomous veterinary diagnosis.
- New hardware beyond the selected ESP32, sensors, camera, and feeder components.
- Individual goat re-identification based on an untrained deep Re-ID model.
- Major UI redesigns that do not address an acceptance criterion or usability defect.

5. Deliverables and Acceptance Criteria

Deliverable	Acceptance criteria	Evidence
Integrated web application	All in-scope modules load, navigation is consistent, permissions are enforced, and no critical errors occur in the demo workflow	Signed smoke-test sheet, screenshots, system-check output
IoT monitoring	Selected ESP32 sends valid readings for at least a 2-hour soak test; invalid ranges are rejected; offline state is visible	Timestamped readings, logs, soak-test record
Automated feeder	Scheduled and manual feed commands actuate the chosen device, duplicate/unsafe commands are prevented, and outcomes are logged	Video, actuation logs, failure/retry test
ML detection	Approved goat and person test sets produce recorded precision/recall results; live demo meets the team-approved operating threshold	Evaluation report, sample detections, model/version record
Alerts and SMS	At least one sensor, security, and feeding event creates the correct in-app alert; configured SMS paths record success/failure without leaking credentials	Test logs, screenshots, redacted SMS evidence
Analytics and reports	Report totals reconcile with source records; supported exports open correctly	Reconciliation sheet, exported samples

Deliverable	Acceptance criteria	Evidence
Marketplace	Inquiry-to-completion workflow succeeds; reservation conflicts and unauthorized access are rejected	Automated tests and UAT script
Deployment package	Fresh setup succeeds from documented prerequisites and environment template; production secrets are externalized	Deployment checklist, clean-install record
Capstone package	Final manuscript, architecture diagrams, user/admin guide, test results, presentation, and demo contingency materials are complete	Document checklist and adviser approval

Release acceptance requires no open Severity 1 or Severity 2 defects. Severity 3 defects may be accepted only with a documented workaround and approval from the Project Lead and Product/Research Lead.

6. Delivery Strategy and Schedule

Work will run in one-week iterations. Each week ends with a demonstrable increment, test evidence, and backlog review.

Week	Dates	Primary outcome	Key work	Exit gate
1	Aug 17-21	Scope and architecture frozen	Confirm requirements, select canonical firmware, map end-to-end flows, inventory hardware/services, establish source control and issue board	Scope, owners, architecture, and demo scenarios approved
2	Aug 24-28	Stable development baseline	Document setup, sanitize configuration, seed test data, add CI checks, create test matrix, resolve environment drift	Clean setup works; checks and current 19 tests pass in repeatable workflow
3	Aug 31-Sep 4	IoT and feeder integration complete	Finish MQTT/command path, device acknowledgement, retries/timeouts, schedule safety, offline behavior, hardware bench tests	Sensor-to-dashboard and command-to-actuator flows pass bench tests
4	Sep 7-11	ML and security workflow validated	Lock models/thresholds, test camera reliability, evaluate detection set, connect person/goat events to alert records	Metrics recorded; detection-to-alert demo is repeatable
5	Sep 14-18	Business workflows integrated	Validate goat records, SMS, analytics reconciliation, marketplace lifecycle, permissions, exports	End-to-end functional test suite passes
6	Sep 21-25	System hardened	Expand automated tests, conduct security/privacy review, backup/restore drill, performance and 2-hour soak tests, resolve high defects	No Severity 1 defects; recovery and soak tests pass
7	Sep 28-Oct 2	User acceptance and documentation complete	Farm-user UAT, usability fixes, installation/operator guides, manuscript evidence, diagrams, video backup	UAT signed; documents reach release-candidate status
8	Oct 5-9	Release and defense ready	Code freeze, regression, clean deployment rehearsal, demo rehearsals, package release, archive evidence	Release checklist signed and final package archived

Critical path

Canonical firmware selection -> sensor/feeder communication -> device acknowledgement and logging -> end-to-end integration testing -> UAT -> code freeze -> final demonstration.

ML evaluation and documentation can proceed in parallel, but the approved model and thresholds must be frozen before full regression testing begins.

7. Work Breakdown and Ownership

Names should replace role labels during the Week 1 kickoff. One person may hold multiple roles on a small capstone team, but every item must have exactly one accountable owner.

Workstream	Accountable role	Main responsibilities
Project governance	Project Lead	Schedule, backlog, decisions, risks, adviser communication, release approval
Requirements and research	Product/Research Lead	Research alignment, success measures, manuscript, UAT approval
Web/backend	Software Lead	Django modules, APIs, data integrity, permissions, background tasks
Frontend/usability	Software Lead	Dashboard consistency, responsiveness, accessibility, operator workflows
IoT and feeder	Hardware/IoT Lead	ESP32 firmware, wiring, calibration, MQTT/HTTP, actuator safety, hardware evidence
ML and cameras	ML Lead	Dataset, model versions, thresholds, camera pipeline, evaluation metrics
QA and release	QA/Documentation Lead	Test plan, regression, defect triage, deployment rehearsal, release archive
Infrastructure	Software Lead	MySQL, Redis/Celery/Channels, environment configuration, backups, observability

RACI for major approvals

Decision or output	Project Lead	Product/Research	Software	Hardware/IoT	ML	QA/Docs
Scope baseline	A	R	C	C	C	C
Architecture and deployment	A	C	R	C	C	C
Hardware readiness	A	C	C	R	C	C
Model and threshold approval	A	C	C	C	R	C
UAT acceptance	C	A/R	C	C	C	R
Release decision	A	C	C	C	C	R

R: Responsible, **A:** Accountable, **C:** Consulted.

8. Product Backlog Priorities

Must complete

1. Select and document one canonical ESP32 firmware build and wiring configuration.
2. Complete and verify the physical feeder command path, including acknowledgement, failure handling, and logs.
3. Create automated tests for feeding safety, authentication/permissions, alerts, SMS failure behavior, and ML API error handling.
4. Establish a reproducible source-control and release-tag process.
5. Define and run a representative ML evaluation set; record model hashes/versions and thresholds.

6. Validate production configuration: `DEBUG=False`, allowed hosts, CSRF/CORS, secrets, static/media handling, database, Redis, Celery, and Channels.
7. Complete end-to-end tests, UAT, backup/restore rehearsal, and deployment rehearsal.
8. Complete operator, installation, troubleshooting, and capstone evidence documents.

Should complete

1. Add device heartbeat/offline alerts and clearer operator recovery actions.
2. Add structured logs and a simple operational health page.
3. Improve accessibility and mobile layouts for core workflows.
4. Automate report-data reconciliation checks.
5. Prepare prerecorded demo paths for camera, network, SMS, or hardware failure.

Could complete after release stability

1. Individual goat recognition using tags or a trained Re-ID model.
2. Advanced predictive analytics.
3. Additional marketplace enhancements.
4. Multi-farm and mobile-app capabilities.

9. Quality Management

Definition of Done

A work item is done only when:

- Its acceptance criteria are met.
- Code and configuration have been peer reviewed.
- Relevant automated and manual tests pass.
- Security, privacy, and failure behavior have been considered.
- User-facing and operational documentation is updated.
- Evidence is attached to the issue or test record.
- No secrets, temporary files, or test credentials are included in the release.

Test levels

Level	Minimum coverage
Static/configuration	Django system check, migration drift check, dependency and secret review
Unit	Validation, scheduling rules, report calculations, model/service helpers
API/integration	Sensor upload, command acknowledgement, authentication, alerts, SMS provider behavior, marketplace lifecycle
Hardware	Calibration, network loss, restart recovery, actuator limits, duplicate commands, feed-level edge cases
ML	Representative labeled dataset, precision, recall, false-positive review, poor-light and occlusion scenarios
System	Sensor -> dashboard -> rule/alert/report and schedule -> feeder -> acknowledgement/log
Non-functional	Security review, backup/restore, response-time check, 2-hour soak test, mobile usability
UAT	Operator executes the approved demo scenarios without developer intervention

Defect severity

- **Severity 1 - Critical:** Safety risk, data loss, credential exposure, system unavailable, or actuator cannot be stopped. Fix immediately; blocks all release work.
- **Severity 2 - High:** Core workflow fails with no acceptable workaround. Fix before release.
- **Severity 3 - Medium:** Partial failure or usability issue with a documented workaround. Triage into the current or next iteration.
- **Severity 4 - Low:** Cosmetic or low-impact improvement. Place in the post-release backlog.

10. Risk Register

ID	Risk	Probability / impact	Mitigation and contingency	Owner	Trigger
R1	Feeder does not actuate reliably or creates a safety risk	Medium / Critical	Add limits, acknowledgement, timeout, manual stop, dry runs, and supervised load tests; keep manual feeding available	Hardware/IoT Lead	Missed, duplicate, or continuous actuation
R2	Wi-Fi, MQTT, Redis, or external service outage breaks the demo	High / High	Local network, cached/local demo data, retry states, offline UI, prerecorded evidence, tested recovery steps	Software Lead	Connectivity loss or repeated timeout
R3	ML false positives/negatives undermine findings	Medium / High	Freeze representative dataset, tune thresholds, report limitations, improve camera placement/lighting	ML Lead	Metric falls below approved threshold
R4	SMS quota, credentials, or provider availability fails	Medium / Medium	Monitor logs/quota, mock provider tests, in-app alert fallback, redact secrets	Software Lead	Provider error or quota warning
R5	Sensitive credentials or farm/person images are exposed	Medium / Critical	Externalize secrets, restrict media access, apply least privilege, define retention/consent, perform release scan	Project Lead	Secret scan hit or unauthorized access
R6	Database or media loss	Low / Critical	Automated backup, off-device copy, restore drill, documented retention	Software Lead	Failed backup or restore validation
R7	Scope growth delays the capstone	High / High	Enforce must/should/could priorities and change control; defer Re-ID and enhancements	Project Lead	New feature affects critical path

ID	Risk	Probability / impact	Mitigation and contingency	Owner	Trigger
R8	Limited automated coverage allows regression	High / High	Add risk-based tests in Weeks 2-6; run full regression at each merge and release candidate	QA/Docs Lead	Core module changed without tests
R9	Multiple firmware variants cause configuration drift	High / Medium	Select one canonical build; archive alternatives; version firmware with wiring/config notes	Hardware/IoT Lead	Different firmware used across tests
R10	Production setup differs from development	Medium / High	Clean-machine rehearsal with production-like database, Redis, Celery, static/media, and environment settings	Software Lead	Setup guide cannot reproduce system
R11	Hardware component fails near demonstration	Medium / High	Maintain spare ESP32/sensor/servo, wiring diagram, known-good firmware binary, and video backup	Hardware/IoT Lead	Unstable readings or physical damage
R12	Team availability or documentation lag	Medium / Medium	Weekly capacity check, early document drafting, shared ownership, evidence captured during testing	Project Lead	Task slips more than two working days

The risk register is reviewed twice weekly. Any Critical-impact risk with increasing probability is escalated to the adviser within one working day.

11. Communication and Control

Cadence

- **Monday, 30 minutes:** iteration planning, capacity, dependencies, risk review.
- **Daily, 10 minutes:** completed work, next task, blockers.
- **Wednesday, 20 minutes:** integration checkpoint for software, IoT, and ML.
- **Friday, 45 minutes:** demo, test results, defect triage, adviser-ready status summary.
- **Weeks 7-8:** at least two complete timed defense rehearsals per week.

Required project records

- Backlog/issue board with owner, priority, acceptance criteria, and target week.
- Decision log for scope, hardware, model, threshold, infrastructure, and release choices.
- Risk and issue register.
- Test matrix and defect log.
- Versioned source repository with protected main branch and tagged releases.
- Evidence folder containing test output, screenshots, exports, evaluation results, and demo video.

Weekly status format

Each report should contain:

1. Overall status: Green, Amber, or Red.
2. Outcomes completed against the weekly exit gate.
3. Planned work for the next week.
4. Top three risks/issues and needed decisions.

5. Test summary and open defects by severity.
6. Schedule variance and recovery action.

12. Change Control

Any proposed scope change must state the user/research value, acceptance criteria, effort, affected milestone, risk, and what existing item will be deferred. The Project Lead records the decision; the Product/Research Lead approves research-scope changes; the relevant technical lead confirms feasibility. Changes affecting the critical path after September 18 require adviser approval. After the October 5 code freeze, only release-blocking defect fixes are allowed.

13. Monitoring Metrics

The team will track:

- Weekly milestone exit gates passed on time.
- Must-have backlog completed versus planned.
- Open defects by severity and average age.
- Automated tests passing and coverage of in-scope modules.
- Sensor message success rate and device uptime during soak testing.
- Feeder command success, acknowledgement time, and duplicate/failure count.
- ML precision, recall, and false-positive count on the frozen evaluation set.
- Alert creation/delivery success and latency.
- Backup success and measured restoration time.
- UAT scenarios passed without assistance.

Target release indicators are 100% of must-have acceptance criteria passed, zero Severity 1/2 defects, 100% feeder safety cases passed, a successful restore drill, a passed 2-hour hardware soak test, and signed UAT.

14. Release and Demonstration Checklist

Before final release:

- ☐ Scope, architecture, and canonical firmware are documented.
- ☐ Production environment variables are set and secrets are excluded from deliverables.
- ☐ Database migrations, static files, media permissions, Redis, Celery, Channels, and scheduled jobs are verified.
- ☐ Full automated regression passes on the release candidate.
- ☐ Hardware, ML, security, recovery, and UAT evidence is approved.
- ☐ Database and media backup can be restored.
- ☐ Demo accounts and anonymized/consented data are prepared.

- [] Live demo script is timed and rehearsed.
- [] Spare hardware, local network, offline data, screenshots, and prerecorded demo are available.
- [] Installation, user, admin, troubleshooting, and maintenance guides are complete.
- [] Final manuscript and presentation use results from the same frozen release.
- [] Release is tagged, checksummed where appropriate, and archived with model and firmware versions.

15. Immediate Next Actions

Due	Action	Owner	Output
Aug 17	Confirm submission/defense date, named team members, adviser, and evaluator expectations	Project Lead	Updated plan header and owner map
Aug 17	Create or verify the Git remote, branch policy, issue board, and release convention	Software Lead	Traceable repository and board
Aug 18	Select the canonical ESP32 firmware and record hardware/wiring/configuration	Hardware/IoT Lead	Approved device baseline
Aug 18	Approve three to five end-to-end demo/UAT scenarios	Product/Research Lead	Signed scenario list
Aug 19	Build the requirements-to-test traceability matrix	QA/Docs Lead	Traceability matrix
Aug 20	Design the feeder MQTT/acknowledgement completion task and safety tests	Hardware/IoT + Software Leads	Implementable story set
Aug 21	Freeze ML models, test dataset, thresholds, and evaluation method	ML Lead	Model evaluation baseline
Aug 21	Review Week 1 exit gate and re-baseline dates if the final deadline differs	Project Lead	Approved Week 2 plan

16. Approval

Role	Name	Signature/date
Project Lead	To be assigned	
Product/Research Lead	To be assigned	
Software Lead	To be assigned	
Hardware/IoT Lead	To be assigned	
ML Lead	To be assigned	
QA/Documentation Lead	To be assigned	
Adviser	To be assigned	